Analysis and Design of Algorithms

CS3230

Prof. Lee Wee Sun

NP-Completeness I

- Polynomial time
- SUBSET-SUM
- Decision problems
- P, NP, co-NP
- Problem Encoding
- Reduction
- NP-Completeness
- CIRCUIT-SAT

Polynomial Time

Consider a problem of size n

A ***polynomial time*** algorithm for the problem has a worst case time complexity of $O(n^c)$ for some constant c. For example

- Linear time: $O(n)$, quadratic time: $O(n^2)$, …

A ***superpolynomial time*** algorithm for the problem has worst case complexity of $\omega(n^c)$ for all constants c. For example:

- Exponential time $\Theta(2^n)$, factorial time $\Theta(n!)$, …

SUBSET-SUM

Input: A set of integers S and an integer t

Output: Decide if there is a non-empty subset $S' \subseteq S$ such that $\sum_{x \in S'} x = t$

Examples:

- $S = \{-3, 9, 20, 21, 37, -43\}$, $t = 0$

  Answer: *Yes* $(-3 + 9 + 37 - 43 = 0)$

- $S = \{-1, -3, -5, 10, 14, 20\}$, $t = 0$

  Answer: *No* (No subset sums to 0)

A Simple Solution …

Brute-force algorithm:

- Test every subset $S' \subseteq S$ to see if $\sum_{x \in S'} x = t$

Running time analysis:

- Suppose $|S| = n$
- 2^n subsets S' of S
- Running time is $\Omega(2^n)$

Is there a polynomial time algorithm?

Decision and Optimization Problems

Decision problem:

Answer is either *YES* or *NO* (also represented as 1 and 0)

- E.g. The SUBSET-SUM problem

Optimization problem:

Output an *optimal value* (and *solution*)

- E.g. The shortest path problem: Given a graph G, two vertices u and v, report a path from u to v with the **fewest** # of edges

Given an optimization problem, we can convert it into a corresponding decision problem

- Ask whether there is a solution with value $\leq k$
- E.g. The shortest path problem: Given a graph G, two vertices u and v, is there a *path* from u to v using no more than k edges?

The decision problem is ***no harder*** than the optimization problem

- Given the value of the optimal solution, check whether it is $\leq k$
  - If we can solve the optimization problem quickly, we can solve the decision problem quickly
  - **If we cannot solve the decision problem quickly, we cannot solve the optimization problem quickly**

To show ***hardness***, enough to study ***decision problems***

Recap:

- Polynomial time and superpolynomial time
- SUBSET-SUM as an example of a difficult problem
- Construct a decision problem from an optimization problem – difficulty of decision problem implies difficulty of optimization problem

Analysis and Design of Algorithms

CS3230

Prof. Lee Wee Sun

NP-Completeness I

- Polynomial time
- SUBSET-SUM
- Decision problems
- **P, NP, co-NP**
- Problem Encoding
- Reduction
- NP-Completeness
- CIRCUIT-SAT

Classes of Problems

- **P:** Problems solvable in polynomial time
- **NP:**

  Problems with polynomial time verifiable solutions
- **co-NP:** Problems with polynomial time verifiable counterexamples

P (Polynomial time)

Problems in **P** can be solved in polynomial time

- ❑ Sorting
- ❑ Shortest path in a graph
- ❑ Minimum spanning tree
- ❑ Matrix multiplication
- ❑ …

NP (Non-deterministic Polynomial)

A problem is in **NP** if polynomial time verifiable ***certificates*** (solutions) of *YES* instances exists.

- There exists a verification algorithm $A(x,y)$ that takes in instances x and certificate y with $|y| = O(|x|^c)$, such that $A(x,y) = 1$ if and only if the answer to x is *YES*.

Example: SUBSET-SUM

Decide if there is a non-empty subset $S' \subseteq S$ such that $\sum_{x \in S'} x = t$

- Certificate: A subset S'
- Verification algorithm checks whether $\sum_{x \in S'} x = t$ in polynomial time

NP stands for ***Non-deterministic Polynomial***

- *Non-deterministic machines* are machines that always guess correctly when there is a choice: *luckiest possible guesser*

A problem is in NP if it is solvable in *polynomial time* by non-deterministic machines – equivalent to earlier definition that the exists polynomial time verifiable *certificates* of *YES* instances exists.

co-NP

A problem is in ***co-NP*** if polynomial time verifiable certificates (*counterexamples*) of *NO* instances exists

Boolean Formula

A Boolean formula $F(X)$ is formed by

1. A set of Boolean variables $X = \{x_1, \ldots x_k\}$
2. Negation symbols $\neg$
3. And symbols $\wedge$
4. Or symbols $\vee$
5. Parenthesis

Example: $F(\{x_1, x_2, x_3\}) = (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_3)$

- If $x_1=1$, $x_2=0$, $x_3=1$, then $F(X) = 0$
- If $x_1=1$, $x_2=0$, $x_3=0$, then $F(X) = 1$

TAUTOLOGY

Input: A Boolean function F(X)

Output: *YES* if $F(X) = 1$ for all assignments to the Boolean variables, *NO* otherwise.

Examples:

- $F(X) = (x_1 \vee \neg x_1) \wedge (x_2 \vee \neg x_2)$
  - Answer: *YES*
- $F(X) = (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_1)$
  - Answer: *NO*

  (Reason: when $x_1 = 0$, $x_2 = 1$, $F(X) = 0$)

Certificate: The assignment X s.t. $F(X)$ evaluates to 0

- The verification algorithm verifies $F(X) = 0$ in polynomial time
- This verifies a *NO* instance – tautology is in **co-NP**

$P \subseteq NP$ and $P \subseteq co\text{-}NP$

Proof: Let X be a problem in P. Let A be a polynomial time algorithm for the X

- Just ignore the certificate
- Run algorithm A in polynomial time, to decide if the answer is *YES* or *NO*

Hence, $P \subseteq (NP \cap co\text{-}NP)$

Integer Factorization (FACTOR)

Input: Two positive integers m and t with $t < m$

Output: *YES* if m is divisible by f where $1 < f \leq t$. Otherwise, *NO.*

Example:

- Instance $(m = 3741,\ t = 5)$
  - *YES* as 3741 is divisible by 3
- Instance $(m = 17017,\ t = 6)$
  - *NO* as $17017 = 7*11*13*17$

No known polynomial time algorithm (in the length of m) for integer factorization

- Public-key cryptography relies on this fact!
- Integer factorization is in both *NP* and *co-NP* (but believed not to be in *P*).

FACTOR is in NP

Certificate: A factor f that divides m

In polynomial time, we can check if:

1. $1 < f \leq t$, and
2. Integer m is divisible by f

Example:

- When (m=3741, t=5), output *YES* since 3741 is divisible by 3

FACTOR is in co-NP

Certificate: The prime factorization of m, i.e., $m = f_1 f_2 \ldots f_k$

By the *fundamental theorem of arithmetic*, the prime factorization always exists and is unique up to the ordering of the factors.

In polynomial time, we can verify that

1. Integer $m = f_1 f_2 \ldots f_k$
2. None of the factors is smaller than t
3. Using AKS primality test, we can verify $f_1, f_2, \ldots, f_k$ are prime numbers in polynomial time

Example: When (m=17017, t=6), output *NO* since 17017 = 7*11*13*17

P=NP?

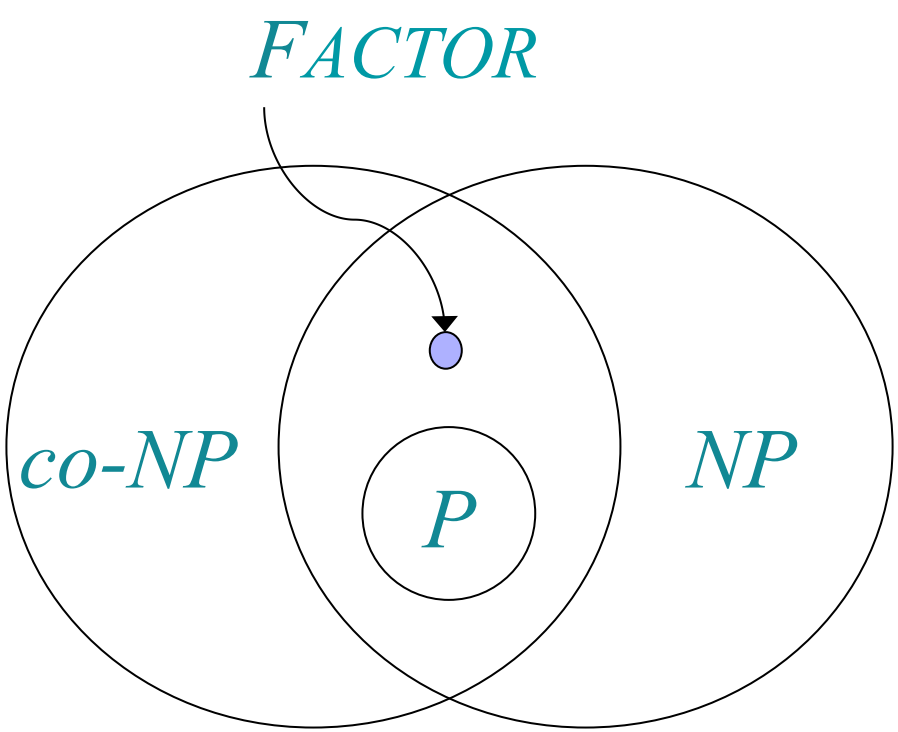
A **Millennium Prize** problem that has been open since 1971 and one of the most important problems in mathematics and theoretical computer science

People generally think that **P≠NP**

If **P=NP**, it will greatly affect the practical world

- It will break many cryptographic solutions, including public-key cryptography
- Many optimization problems, including travelling salesman, integer programming, etc. becomes easily solvable

Likely Relationship among Classes



Known:

- $P \subseteq (NP \cap co\text{-}NP)$
- $FACTOR \in (NP \cap co\text{-}NP)$

Hypothesized:

- $P \neq NP$
- $NP \neq co\text{-}NP$
- $FACTOR \notin P$

Analysis and Design of Algorithms

CS3230

Prof. Lee Wee Sun

NP-Completeness I

- Polynomial time
- SUBSET-SUM
- Decision problems
- P, NP, co-NP
- **Problem Encoding**
- Reduction
- NP-Completeness
- CIRCUIT-SAT

FACTOR

Input: Two positive integers m and t with $t < m$

Output: *YES* if m is divisible by f where $1 < f \leq t$. Otherwise, *NO.*

Why is this **not** a polynomial-time algorithm for solving FACTOR?

```
for f = 2 to t
    if m % f == 0,
        then return true
return false
```

- Problem encoding
  - $\Theta(\lg m)$ bits to encode m
  - $\Theta(\lg t)$ bits for t where $\lg t = \Theta(\lg \mathrm{m})$
  - Total encoding length is $l = \Theta(\lg \mathrm{m})$
- Runtime is $\Omega(m) = \Omega(a^l)$, for some a, so exponential in encoding length l

- For polynomial time, we mean that the runtime is polynomial in the *length of the problem encoding*.
- For many problems can use a "standard" encoding
  - Binary encoding of integers
  - For mathematical objects: list of parameters enclosed in braces, separated by commas.

- Complexity results hold for any encoding that can be transformed to and from the standard encoding in polynomial time, e.g.
  - Changing the encoding of integers from base 2 to base 10
  - Graph: Adjacency matrix is a n by n matrix where entry (i, j) is 1 if $(v_i, v_j) \in E$ and 0 otherwise. Adjacency list contains n lists, where list i contains all vertices j where $(v_i, v_j) \in E$

Pseudo-polynomial time algorithms

An algorithm that runs in time polynomial in the numeric value of the input but is exponential in the length of the input is called a **pseudo-polynomial** time algorithm.

The factoring algorithm shown earlier is a pseudo-polynomial time algorithm.

Analysis and Design of Algorithms

CS3230

Prof. Lee Wee Sun

NP-Completeness I

- Polynomial time
- SUBSET-SUM
- Decision problems
- P, NP, co-NP
- Problem Encoding
- **Reduction**
- NP-Completeness
- CIRCUIT-SAT

Polynomial Time Reduction

Consider two problems A and B:

A can be solved as follows:

Input: An instance α of A

1. Convert α to an instance β of B
2. Solve β and obtain a solution
3. Based on the solution of β, we obtain the solution of α

If steps **1** and **3** take polynomial time, we say $A \leq_P B$

Matrix Multiplication and Squaring

MAT-MULTI

Input:

- Two n x n matrices A and B

Output:

- A x B

MAT-SQR

Input:

- One n x n matrix C

Output:

- C^2

MAT-SQR $\leq_P$ MAT-MULTI

Recall: $A \leq_p B$ means that problem A is reducible to problem B in polynomial time

Reduction: *MAT-SQR* $\leq_p$ *MAT-MULTI*

Assign $A=C$ and $B=C$, run *MAT-MULTI* to get AB which corresponds to C^2

MAT-MULTI $\leq_P$ MAT-SQR

Input: A and B

Reduction: *MAT-MULTI* $\leq_p$ *MAT-SQR*

Construct:

$$C = \begin{bmatrix} 0 & A \\ B & 0 \end{bmatrix}$$

Call MAT-SQR to get

$$C^2 = \begin{bmatrix} 0 & A \\ B & 0 \end{bmatrix} \begin{bmatrix} 0 & A \\ B & 0 \end{bmatrix} = \begin{bmatrix} AB & 0 \\ 0 & BA \end{bmatrix}$$

Reduction between Decision Problems

Consider two decision problems A and B

- Let α be an instance of the decision problem A
- Let β be an instance of the decision problem B

A ***polynomial time reduction*** from A to B ($A \leq_P B$) is a transformation with the following properties:

- The transformation takes *polynomial time*
- The answer for α is *YES* if and only if the answer for β is *YES*

Suppose $A \leq_P B$

Implications:

- If B can be solved in polynomial time, so can A
- If A is hard, so is B

NP-Hard

A problem $L \in$ ***NP-hard*** if any other problem $L' \in NP$ can be reduced to L in polynomial time (i.e. $L' \leq_P L$ *for all* $L' \in NP$).

Implication:

If L can be solved in polynomial time, then ***ALL*** problems in NP can be solved in polynomial time

NP-Complete

If $L \in NP\text{-}hard$ and $L \in NP$, then $L \in$ ***NP-complete (NPC)***

Hardest problem in NP!

Example:

SUBSET-SUM is NP-complete

If a problem is NP-complete, it is generally believed to be ***intractable.***

Recap:

- Polynomial time reduction as a tool to show that one problem is as difficult as another
- NP-hardness
- NP-Completeness

Analysis and Design of Algorithms

CS3230

Prof. Lee Wee Sun

NP-Completeness I

- Polynomial time
- SUBSET-SUM
- Decision problems
- P, NP, co-NP
- Problem Encoding
- Reduction
- NP-Completeness
- **CIRCUIT-SAT**

NP-Completeness

If a problem L is NP-complete, it is generally believed to be ***intractable***

- If L can be solved in polynomial time, all problems in NP can be solved in polynomial time

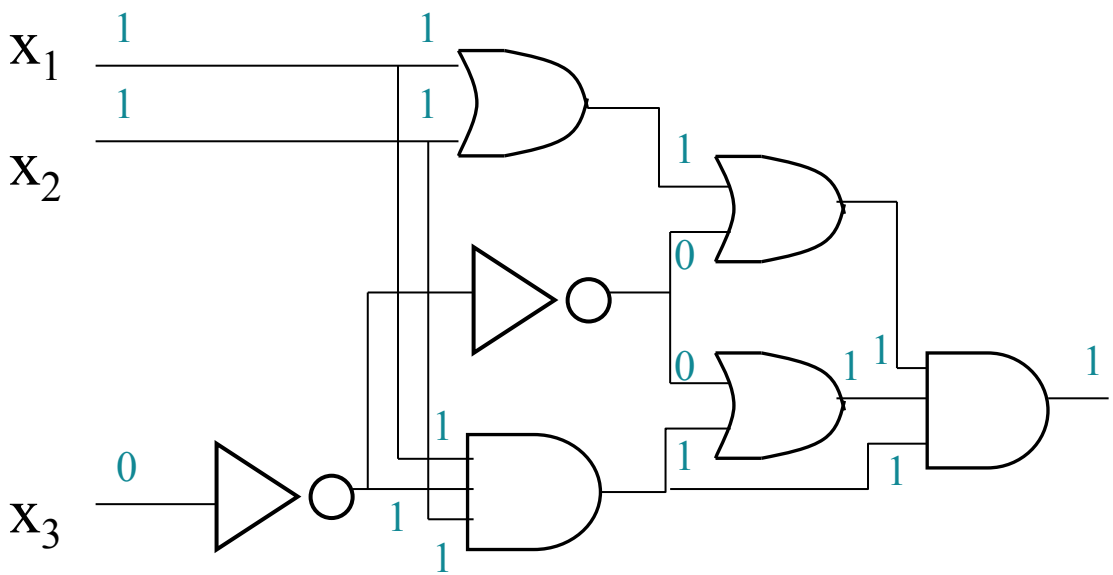
A problem $L \in$ ***NP-complete (NPC)*** if it is in NP and any other problem $L' \in NP$ can be reduced to L in polynomial time (i.e. $L' \leq_P L$ *for all* $L' \in NP$).

Is there any such problem?

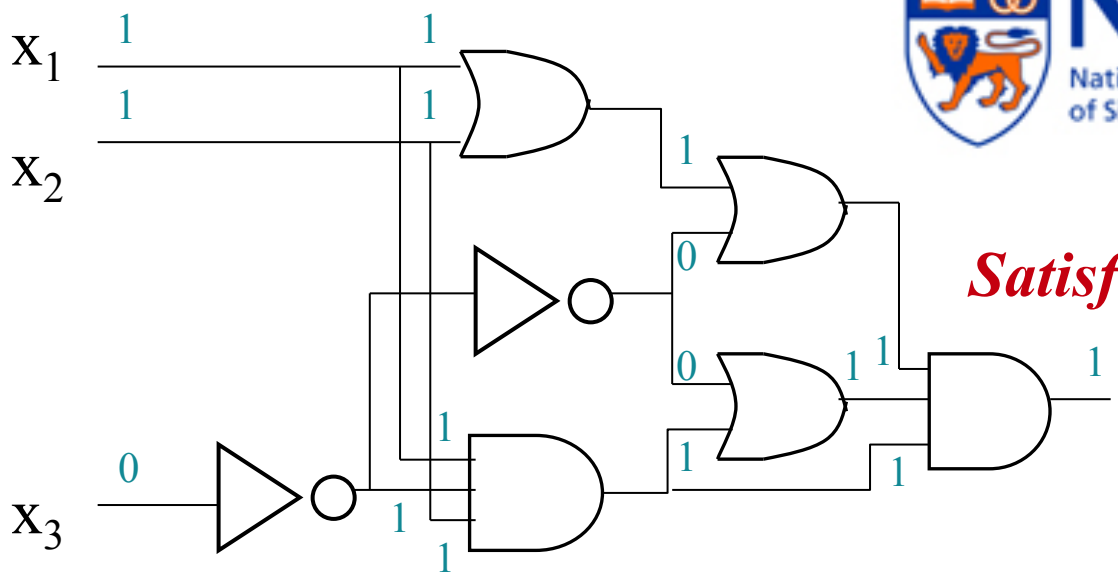
CIRCUIT-SAT is NP-Complete

Input: A Boolean circuit composed of logic gates **AND**, **OR**, and **NOT** with single output

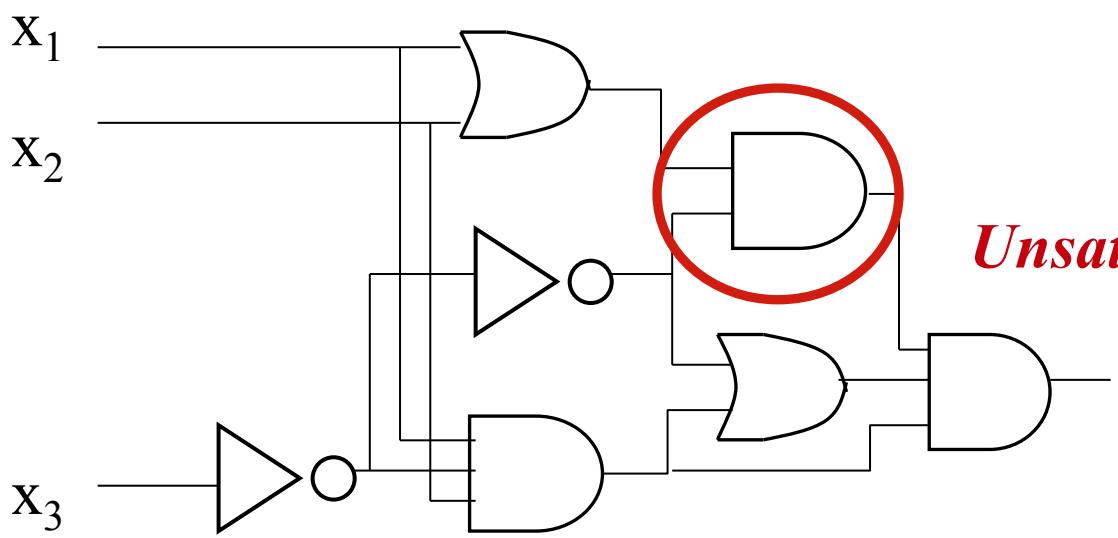
Output: Does there exist an assignment of the input variables so that the circuit is **satisfiable**?



- A satisfying assignment is a truth assignment that causes the output to be 1
- A circuit is satisfiable if it has a satisfying assignment
  - $(x_1=1, x_2=1, x_3=0)$ is a satisfying assignment



Satisfiable

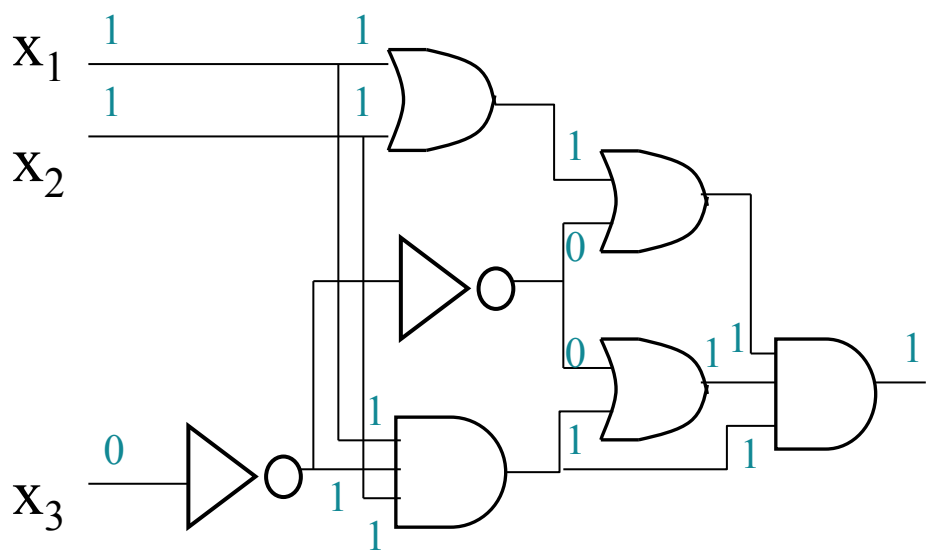


Unsatisfiable

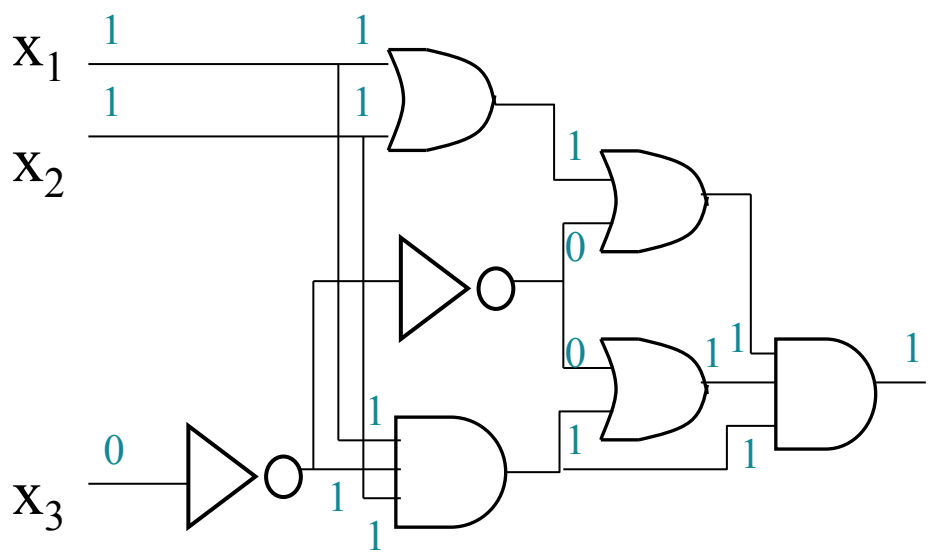
CIRCUIT-SAT is in NP

Proof: CIRCUIT-SAT is polynomial time verifiable

- **Certificate:** The assignment of input values and output value for every gate in C (i.e. the values of the wires)



- We check in polynomial (linear in # of gates) time for every gate:
  - the computed output value matches the value of the output wire given in the certificate
  - the output of the whole circuit is 1



CIRCUIT-SAT is NP-hard

Main idea:

For any problem $L \in NP$, show that
$L \leq_P$ CIRCUIT-SAT

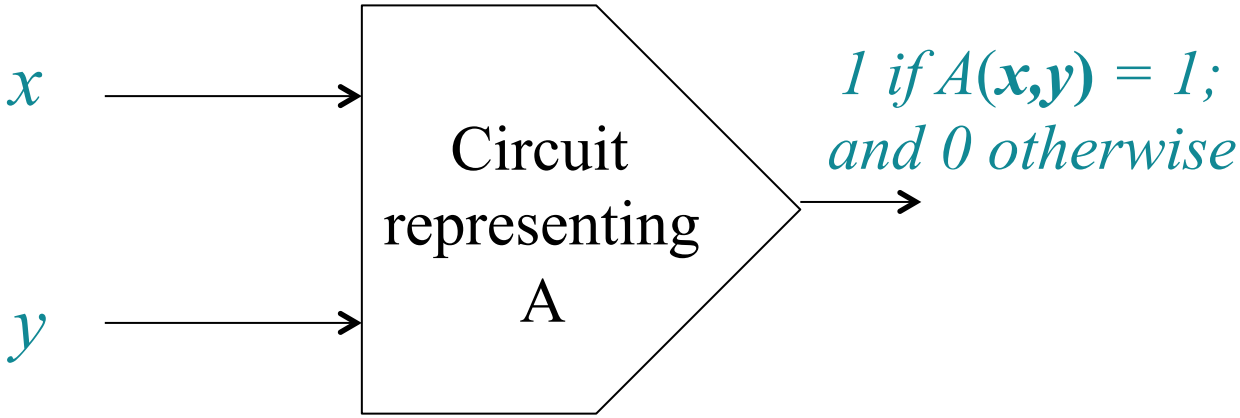
In another word, we need a polynomial time algorithm f that maps any instance $x \in L$ to a circuit $C=f(x)$ such that the answer to x is *YES if and only if* $C=f(x)$ is *satisfiable*

A Verifier for L Exists

Since $L \in NP$, there exists a verification program A such that

- For any instance $x \in L$, $A(x, y) = 1$ if and only if y can be used to verify that the answer to x is yes
- A runs in polynomial time

We will build a circuit to simulate the execution of the computer program A(x,y)



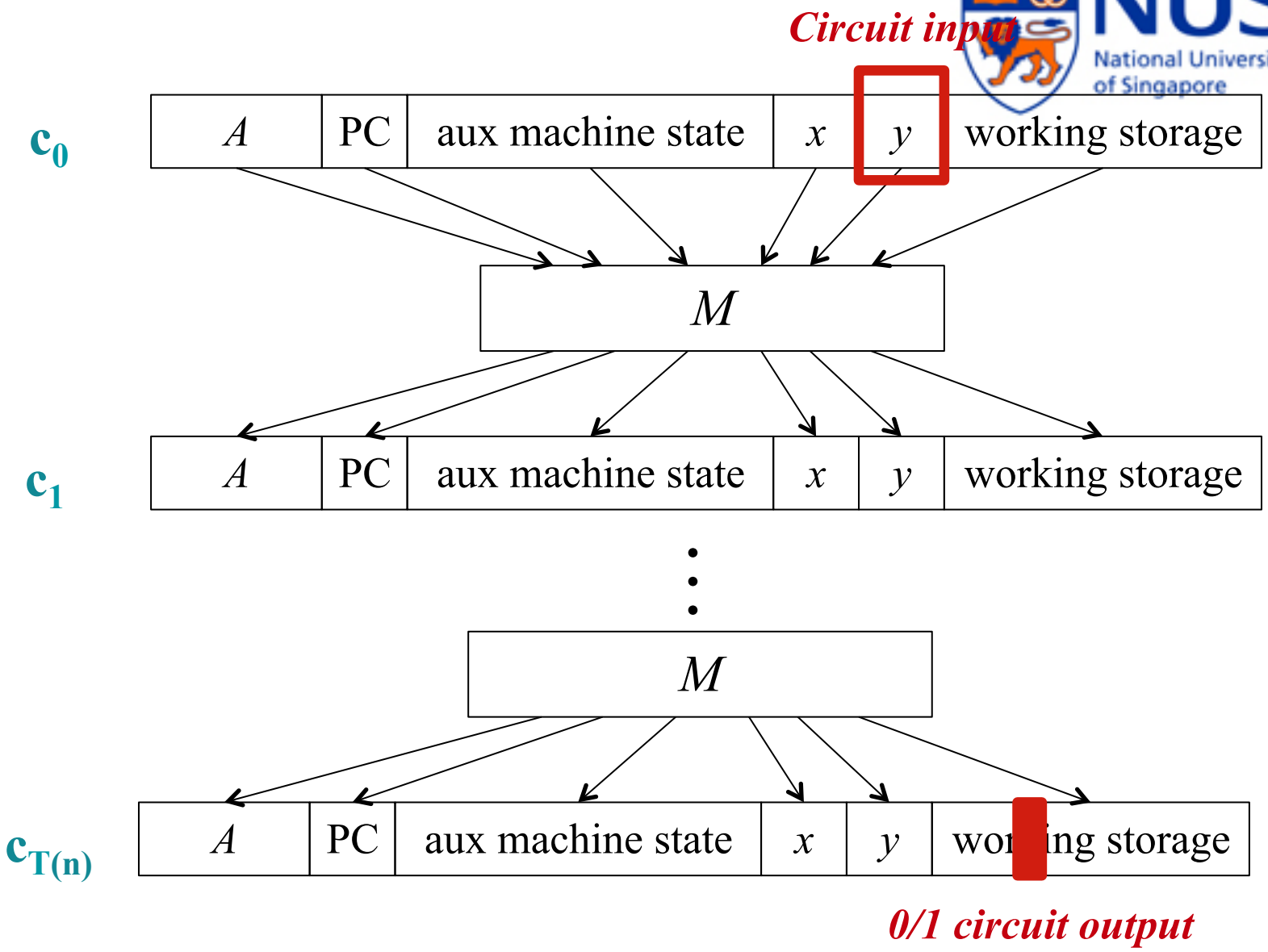
Simply map the execution of A(x,y) on a computer onto a circuit

The computation of A is viewed as a sequence of configurations inside a computer which represent the state of computation at every clock cycle: $c_0, c_1, \ldots, c_i, c_{i+1}, \ldots, c_{T(n)}$ where $n = |x|$ and each c_i has

- Program A, program counter PC, auxiliary machine state, the input x, the certificate y, working storage

Configuration c_i is mapped to c_{i+1} by the combinational circuit M that implements the computer hardware

The algorithm A runs for at most $T(n)$ steps, then the output of algorithm A appears as one bit in configuration $c_{T(n)}$


Circuit input
c_0
A
PC
aux machine state
x
y
working storage
M
c_1
A
PC
aux machine state
x
y
working storage
M
$c_{T(n)}$
A
PC
aux machine state
x
y
working storage
0/1 circuit output

For each c_i:

- A is a program that has a constant size;
- PC is constant size
- *aux machine state* has constant size
- x is of length n
- the certificate y is of length *polynomial* in n
- *working storage* have size *polynomial* in n

The sub-circuit M between c_i and c_{i+1} is of polynomial size

The verification algorithm A runs in $T(n)$ steps where T is polynomial.

Hence, the whole circuit is of polynomial size (at most $T(n)$ copies of M) and can be constructed in polynomial time

If the solution for x is YES, circuit C is satisfiable:

If the answer to x is *YES* there exists a certificate y such that $A(x, y) = 1$.

As circuit $C = f(x)$ is a *simulation of verification program* A, it will give $A(x, y) = 1$ if given certificate y

Hence, the circuit $C = f(x)$ is satisfiable (with input y)

If circuit C is satisfiable, the solution for x is YES

If $C = f(x)$ is satisfiable, there exists some y such that the output of circuit $f(x)$ is 1

This implies that $A(x, y) = 1$ because $C = f(x)$ is the *simulation of verification program* A

Hence the answer to problem instance x is *YES*

CIRCUIT-SAT is NP-Complete

CIRCUIT-SAT is NP as it is *verifiable in polynomial time*

CIRCUIT-SAT is NP-hard as ***every*** *NP problem can be reduced to CIRCUIT-SAT in polynomial time*

Thus CIRCUIT-SAT is *NP-complete*